🌱 Spring Framework

Core Basics & IoC Container

Complete Interview Notes — From Zero to Clear Concepts

📚 Topics Covered

What is Spring • Why Spring • Spring Modules
IoC Principle • IoC Container • BeanFactory vs ApplicationContext
Bean Definition • Bean Scopes • Bean Lifecycle
Dependency Injection — 3 Types • @Autowired • Wiring Modes
Configuration Styles • @Annotations • Interview Q&A

🎯 Interview-Ready | Simple Language | Real Examples | Visual Flows

1️⃣ What is Spring Framework?

Spring is a Java framework that makes it easy to build enterprise applications. Before Spring, Java developers had to write a LOT of repetitive setup code (boilerplate). Spring removes that pain.

🎭 **Real-World Analogy**

🏠 Building a house WITHOUT Spring = buying land + bricks + cement + doing everything yourself.

🏠 Building WITH Spring = moving into a fully furnished flat — kitchen, electricity, plumbing ready!

Spring provides ready-made infrastructure. You focus on YOUR business logic only.

🔧 Problems Spring Solves

Problem Before Spring	How Spring Solves It
Tight Coupling	Classes create their own dependencies (new Obj()). Hard to test & change.
Boilerplate Code	Repetitive setup code for DB connections, transactions, etc.
Testability	When you create objects yourself, you can't swap them with mocks in tests.
Configuration Hell	Old Java EE needed complex XML (web.xml, ejb-jar.xml, etc.)

💡 Spring's Core Philosophy

- Inversion of Control (IoC) — 'Don't call us, we'll call you'
- Dependency Injection (DI) — Spring creates and connects objects for you
- POJO-Based — Plain Old Java Objects, no framework-specific base classes needed
- Modularity — Use only what you need (no all-or-nothing)

❓ **Interview Q:** What is Spring Framework? Why was it created?

✅ Spring = lightweight, open-source Java framework for enterprise apps.
✅ Created by Rod Johnson in 2003 as an alternative to heavy Java EE (EJB).
✅ Main goals: Simplify development, remove boilerplate, make code testable.
✅ Core features: IoC Container, DI, AOP, Transaction Management, MVC, Security.
✅ Key principle: YOU write business logic, Spring handles the infrastructure.

2️⃣ Spring Modules — The Big Picture

Spring is NOT one big thing. It's a collection of modules. You pick only what you need.

```
╔═══════════════════════════════════════════════════════════════╗
║               SPRING FRAMEWORK ARCHITECTURE                   ║
╟═══════════════════════════════════════════════════════════════╢
 ║  [ Spring Core ]   [ Spring Beans ]   [ Spring Context ] ║
 ║           ← These are the FOUNDATION (IoC/DI) →          ║
╟═══════════════════════════════════════════════════════════════╢
  ║  [ Spring AOP ]   [ Spring Aspects ]                    ║
 ║           ← Cross-cutting concerns (logging, tx) →       ║
╟═══════════════════════════════════════════════════════════════╢
  ║  [ Spring JDBC ]  [ Spring ORM ]  [ Spring TX ]         ║
 ║           ← Data Access Layer →                          ║
╟═══════════════════════════════════════════════════════════════╢
  ║  [ Spring MVC ]   [ Spring WebFlux ]                    ║
 ║           ← Web Layer →                                  ║
╟═══════════════════════════════════════════════════════════════╢
  ║  [ Spring Security ]  [ Spring Data ]  [ Spring Boot ]  ║
 ║           ← Higher-level Projects →                      ║
╚═══════════════════════════════════════════════════════════════╝
```

Module	What It Does
spring-core	IoC and DI fundamentals — the heart of Spring
spring-beans	BeanFactory — creates and manages beans (objects)
spring-context	ApplicationContext — advanced IoC container
spring-aop	Aspect-Oriented Programming (cross-cutting logic)
spring-jdbc	Simplifies JDBC — no more boilerplate try-catch-finally
spring-orm	Integration with JPA, Hibernate, MyBatis
spring-tx	Declarative transaction management (@Transactional)
spring-webmvc	Build web apps and REST APIs (DispatcherServlet)
spring-test	Testing support — MockMvc, @SpringBootTest

3️⃣ IoC — Inversion of Control

IoC is the most important concept in Spring. Everything else builds on it.

🎭 **Real-World Analogy**

🍕 WITHOUT IoC: You (the developer) drive to the grocery store, buy ingredients, cook food yourself.

🍕 WITH IoC: You call Swiggy (Spring). Swiggy brings the food. You just eat (use the object).

🏢 Another way: 'Hollywood Principle' — Don't call us, we'll call you.
Traditional: Your code creates objects (you call the framework).
IoC: Framework creates objects and calls YOUR code.

🔄 IoC: Before vs After

```
// ❌  WITHOUT IoC — Tight Coupling
public class OrderService {
    // YOU create the dependency yourself
    private PaymentService paymentService = new PaymentService();
    private EmailService emailService = new EmailService();
    // Problem: if PaymentService constructor changes → OrderService breaks!
    // Problem: You CANNOT swap PaymentService with MockPaymentService in tests
}
```

```
// ✅  WITH IoC — Loose Coupling
public class OrderService {
    private PaymentService paymentService;  // just declare
    private EmailService emailService;
    // Spring creates these objects and INJECTS them here
    // You can easily swap PaymentService with MockPaymentService in tests!
}
```

⚙️ What IoC Container Does

1. Reads configuration (XML / annotations / Java config)
2. Creates all the bean (object) instances
3. Figures out which bean depends on which
4. Injects dependencies (wires beans together)
5. Manages the full lifecycle of every bean
6. Destroys beans when application shuts down

❓ **Interview Q:** What is IoC? Explain with a real-world example.

✅ IoC = Inversion of Control. Control of creating and wiring objects is transferred from YOUR code to Spring.

✅ Traditional: your code does 'new PaymentService()'. With IoC, Spring creates it and gives it to you.

✅ Real analogy: A restaurant (Spring) prepares the meal. You (developer) just place an order (declare dependency).

✅ Benefit 1 — Loose coupling: OrderService doesn't care HOW PaymentService is created.

✅ Benefit 2 — Testability: Swap real PaymentService with a Mock easily.

✅ Benefit 3 — Maintainability: Change implementation without touching all callers.

4️⃣ IoC Container — Spring's Object Factory

The IoC Container is the core of Spring. It creates, configures, and manages objects (called Beans). Think of it as a big factory that manufactures and stores all your objects.

🎭 **Real-World Analogy**

🏭 IoC Container = A big factory/warehouse.

You give it a blueprint (config) — it manufactures objects (beans).

It stores them, connects them together, and delivers them when needed.

When the factory shuts down — it destroys all the objects cleanly.

📦 Two Types of IoC Container

Container Type	What It Is
BeanFactory	Basic IoC container. Lazy initialization (creates beans only when asked). Lightweight. Use in resource-constrained environments.
ApplicationContext	Advanced IoC container. Extends BeanFactory. Eager initialization. Adds: Events, i18n, AOP, @PostConstruct. Use this in all Spring Boot apps.

⚖️ BeanFactory vs ApplicationContext — Key Differences

Feature	Comparison
Initialization	BeanFactory: Lazy (bean created when first requested) | ApplicationContext: Eager (all beans created at startup)
Event Handling	BeanFactory: Not supported | ApplicationContext: Supports ApplicationEvent
Internationalization	BeanFactory: No | ApplicationContext: Yes (MessageSource)
AOP Integration	BeanFactory: Manual | ApplicationContext: Auto-integrated
@PostConstruct / @PreDestroy	BeanFactory: Not supported | ApplicationContext: Fully supported
Use in Spring Boot	BeanFactory: Never (low-level) | ApplicationContext: Always

🔧 ApplicationContext Implementations

Implementation	When to Use
ClassPathXmlApplicationContext	Loads XML config from classpath. Old style. Rarely used now.
FileSystemXmlApplicationContext	Loads XML config from file system path.
AnnotationConfigApplicationContext	Loads Java @Configuration classes. Modern style.
WebApplicationContext	For web apps. Knows about Servlet context.
SpringApplication.run()	Spring Boot's way — auto-detects and creates the right context.

```
// Old XML-based (rarely used now)
ApplicationContext ctx = new ClassPathXmlApplicationContext("beans.xml");

// Modern Java config based
ApplicationContext ctx = new AnnotationConfigApplicationContext(AppConfig.class);

// Spring Boot (most common — auto-creates context)
ApplicationContext ctx = SpringApplication.run(MyApp.class, args);

// Getting a bean from the container
OrderService orderService = ctx.getBean(OrderService.class);
OrderService os2 = (OrderService) ctx.getBean("orderService"); // by name
```

❓ **Interview Q:** What is the difference between BeanFactory and ApplicationContext?

✅ Both are IoC containers. ApplicationContext extends BeanFactory (is a superset).
✅ BeanFactory: Lazy — creates bean only when ctx.getBean() is called.
✅ ApplicationContext: Eager — creates ALL singleton beans at startup.
✅ ApplicationContext adds: Event publishing, i18n support, AOP, @PostConstruct/@PreDestroy.
✅ In practice: ALWAYS use ApplicationContext. BeanFactory is too low-level.
✅ Spring Boot automatically creates an ApplicationContext for you.

5️⃣ Spring Beans — Objects Managed by Spring

A Bean is any Java object that is created, configured, and managed by the Spring IoC Container. In simple words: if Spring knows about an object, it is a Bean.

🎭 **Real-World Analogy**

🍲 Think of the IoC Container as a big canteen kitchen.
Beans = the food items being prepared in the kitchen.
The kitchen (container) creates them, stores them, and serves them on demand.
You don't cook (new RestaurantService()) — you just request from the kitchen!

📝 How to Define a Bean — 3 Ways

Way 1: XML Configuration (Old Style)

```
<!-- beans.xml -->
<beans>
    <bean id="restaurantService" class="com.app.RestaurantService">
        <property name="repository" ref="restaurantRepository"/>
    </bean>
    <bean id="restaurantRepository" class="com.app.RestaurantRepository"/>
</beans>
```

Way 2: Annotation-Based (Modern — Most Common)

```java
// Just add a stereotype annotation — Spring auto-detects via @ComponentScan
@Service                    // tells Spring: 'I am a bean, manage me!'
public class RestaurantService {
    @Autowired              // tells Spring: 'inject a RestaurantRepository here'
    private RestaurantRepository repository;
}

@Repository
public class RestaurantRepository { }

// Enable scanning in config
@Configuration
@ComponentScan(basePackages = "com.app")
public class AppConfig { }
```

Way 3: Java @Configuration (Explicit Bean Declaration)

```java
@Configuration
public class AppConfig {

    @Bean                   // method name becomes bean ID: 'paymentService'
    public PaymentService paymentService() {
        return new PaymentService("RAZORPAY");  // with constructor args
    }

    @Bean
    public OrderService orderService() {
        return new OrderService(paymentService()); // Spring handles the wiring
    }
}
```

Config Style	When to Use
XML config	Verbose but explicit. Good for third-party classes you can't annotate.
@Component / @Service / @Repository	Cleanest for YOUR classes. Auto-scanned.
@Bean in @Configuration	Best for third-party libraries, or when you need custom construction logic.

❓ **Interview Q:** What is a Spring Bean? How many ways can you declare one?

✅ A Spring Bean = any Java object managed by the Spring IoC Container.
✅ 3 ways to declare a bean:
1. XML: <bean id='...' class='...'/> (legacy, avoid in new projects)
2. Stereotype annotations: @Component, @Service, @Repository, @Controller + @ComponentScan
3. @Bean method inside @Configuration class (for third-party or complex setup)

✅ Spring Boot auto-enables @ComponentScan so annotations work out of the box.

6️⃣ Bean Scopes

Scope = How many instances of the bean Spring creates. By default Spring creates ONE shared instance.

📏 All Bean Scopes

Scope	Behavior
singleton (DEFAULT)	ONE instance per IoC container. All beans share the same object. Most beans should be singleton.
prototype	NEW instance created every time the bean is requested (getBean() or @Autowired).
request	ONE instance per HTTP request. Only in web apps (Spring MVC).
session	ONE instance per HTTP session. Only in web apps.
application	ONE instance per ServletContext (whole web app lifetime).
websocket	ONE instance per WebSocket session.

```
// Singleton (default — no annotation needed)
@Service
public class RestaurantService { }

// Prototype — new object every time
@Component
@Scope("prototype")          // or ConfigurableBeanFactory.SCOPE_PROTOTYPE
public class ShoppingCart { }

// Request scope — web apps only
@Component
@Scope(value = "request", proxyMode = ScopedProxyMode.TARGET_CLASS)
public class RequestContext { }
```

🎭 **Real-World Analogy**

☕ Singleton Scope = The canteen has ONE coffee machine. Everyone uses the SAME machine.

📋 Prototype Scope = Each customer gets a NEW order form (fresh copy every time).

🌐 Request Scope = Each HTTP request gets its own notepad. Thrown away after response.

🔐 Session Scope = Each logged-in user has their own locker. Exists until they log out.

❓ **Interview Q:** What is Singleton scope? Is it thread-safe?

✅ Singleton = Spring creates ONE instance and shares it across the entire application.

✅ It is the DEFAULT scope — you don't need any annotation.

✅ Thread Safety: Singleton beans are NOT automatically thread-safe.

If the bean has mutable STATE (instance variables that change), it can cause issues.

Solution: Singleton beans should be STATELESS (no mutable instance variables).
✅ Service, Repository, Controller beans are stateless → safe as singletons.
✅ ShoppingCart, UserPreference (stateful) → use Prototype or Session scope.

7️⃣ Bean Lifecycle — Birth to Death of a Bean

Spring manages the complete lifecycle of every bean — from creation to destruction. Understanding this is very important for interviews.

```
╔═══════════════════════════════════════════════════════════╗
║                    BEAN LIFECYCLE FLOW                    ║
╟═══════════════════════════════════════════════════════════╢
║  1. Container starts → reads configuration               ║
║        ↓                                                 ║
║  2. Instantiate Bean → calls constructor                 ║
║        ↓                                                 ║
║  3. Populate Properties → @Autowired dependencies injected║
║        ↓                                                 ║
║  4. BeanNameAware / BeanFactoryAware callbacks           ║
║        ↓                                                 ║
║  5. BeanPostProcessor.postProcessBeforeInitialization()  ║
║        ↓                                                 ║
║  6. @PostConstruct (your custom init code runs here)     ║
║        ↓                                                 ║
║  7. InitializingBean.afterPropertiesSet()                ║
║        ↓                                                 ║
║  8. BeanPostProcessor.postProcessAfterInitialization()   ║
║        ↓                                                 ║
║  9. Bean is READY for use ✅                              ║
║        ↓   (used by application...)                      ║
║ 10. Container shuts down                                 ║
║        ↓                                                 ║
║ 11. @PreDestroy (your custom cleanup code)               ║
║        ↓                                                 ║
║ 12. DisposableBean.destroy()                             ║
║        ↓                                                 ║
║ 13. Bean DESTROYED ❌                                    ║
╚═══════════════════════════════════════════════════════════╝
```

💻 @PostConstruct and @PreDestroy Example

```
@Service
public class DatabaseConnectionService {

    private Connection connection;

    // Runs AFTER bean is created and all @Autowired fields are injected
```

```
    @PostConstruct
    public void init() {
        System.out.println("Initializing DB connection pool...");
        connection = createConnectionPool();
        // Good for: DB connections, cache loading, starting schedulers
    }

    // Runs BEFORE bean is destroyed (app shutdown)
    @PreDestroy
    public void cleanup() {
        System.out.println("Closing DB connections...");
        connection.close();
        // Good for: releasing resources, flushing caches, closing sockets
    }
}
```

📄 Custom Init and Destroy (Old XML Way)

```
<!-- XML way — still valid -->
<bean id="dbService" class="com.app.DatabaseService"
      init-method="init" destroy-method="cleanup" />

// Java @Bean way
@Bean(initMethod = "init", destroyMethod = "cleanup")
public DatabaseService databaseService() {
    return new DatabaseService();
}
```

❓ **Interview Q:** Explain the Spring Bean Lifecycle. What is @PostConstruct?

✅ Lifecycle: Instantiate → Inject Dependencies → @PostConstruct → Bean Ready → @PreDestroy → Destroy

✅ @PostConstruct: runs AFTER all @Autowired injections are done. Good for initialization logic.

✅ @PreDestroy: runs BEFORE bean is destroyed. Good for cleanup (close connections, release resources).

✅ Why not put init code in constructor?
Because @Autowired fields are NOT yet injected when constructor runs!
@PostConstruct guarantees all dependencies are already available.

8️⃣ Dependency Injection — 3 Types in Detail

DI is HOW IoC is implemented. Spring uses DI to inject the required objects into a bean.

🎭 **Real-World Analogy**

📦 DI = Spring is the delivery boy. You just say what you need (declare dependency).
Spring creates it and delivers it to your door (injects it).
You never go to the store yourself (no 'new SomeService()').

🏆 Type 1: Constructor Injection ✅ RECOMMENDED

```
@Service
public class OrderService {
    private final PaymentService paymentService;  // final = immutable
    private final EmailService emailService;

    // @Autowired is optional when there's exactly ONE constructor (Spring 4.3+)
    @Autowired
    public OrderService(PaymentService paymentService, EmailService emailService) {
        this.paymentService = paymentService;
        this.emailService = emailService;
    }
}

// Cleaner with Lombok @RequiredArgsConstructor
@Service
@RequiredArgsConstructor       // Generates constructor for all 'final' fields
public class OrderService {
    private final PaymentService paymentService;
    private final EmailService emailService;
}
```

💡 **Remember:** Constructor Injection is the BEST. Use it always. It makes dependencies explicit, allows 'final' fields, and is easy to unit test (just pass mock in constructor).

🔧 Type 2: Setter Injection

```
@Service
public class NotificationService {
    private EmailService emailService;

    @Autowired
    public void setEmailService(EmailService emailService) {
        this.emailService = emailService;
    }
    // Use case: Optional dependencies (not always required)
    // @Autowired(required = false) — Spring won't fail if bean not found
}
```

⚠️ Type 3: Field Injection ❌ AVOID in Production

```
@Service
public class RestaurantService {
    @Autowired                      // Spring injects directly into field
    private RestaurantRepository repository;

    // Problems:
    // 1. Cannot make 'final' (mutable reference)
    // 2. Hard to unit test — need Spring context or reflection hacks
    // 3. Hidden dependencies — not visible in constructor signature
}
```

📊 Comparing All 3 Types

Injection Type	Recommendation
Constructor Injection	✅ BEST. Immutable (final). Testable. Explicit. Use always.
Setter Injection	⚠️ OK for optional dependencies. Fields not final. Less common.
Field Injection	❌ AVOID. Hard to test. Hidden dependencies. No 'final'. Looks clean but has issues.

❓ **Interview Q:** What are the 3 types of Dependency Injection? Which is best and why?

✅ 3 types: Constructor Injection, Setter Injection, Field Injection.
✅ Best: Constructor Injection. Reasons:
1. Dependencies are EXPLICIT (visible in constructor signature)
2. Fields can be 'final' → immutable, thread-safe
3. Object is ALWAYS in valid state after construction
4. Easy to unit test — just pass mock objects in constructor

✅ Field Injection looks clean but is problematic — avoid in production code.
✅ Setter Injection is OK for optional dependencies only.

9️⃣ @Autowired — How Spring Finds the Right Bean

@Autowired tells Spring: 'Please inject the right bean here.' But what if there are multiple beans of the same type? Let's understand how Spring resolves it.

🔍 Autowiring Resolution Process

7. Spring looks for a bean of the EXACT TYPE requested
8. If exactly ONE found → inject it ✅
9. If ZERO found → throw NoSuchBeanDefinitionException ❌
10. If MULTIPLE found → try to match by FIELD NAME (as bean ID)
11. If still ambiguous → throw NoUniqueBeanDefinitionException ❌
12. Fix ambiguity using @Qualifier or @Primary

🎯 @Qualifier — Choose a Specific Bean

```
// Two implementations of the same interface
@Service("razorpayService")
public class RazorpayPaymentService implements PaymentService { }

@Service("stripeService")
public class StripePaymentService implements PaymentService { }

// In the consumer — tell Spring which one to inject
@Service
public class OrderService {
    @Autowired
```

```
    @Qualifier("razorpayService")   // specify exactly which bean
    private PaymentService paymentService;
}
```

⭐ @Primary — Mark One as the Default

```
@Service
@Primary                // This is the DEFAULT when multiple beans of same type exist
public class RazorpayPaymentService implements PaymentService { }

@Service
public class StripePaymentService implements PaymentService { }

// Now @Autowired will pick RazorpayPaymentService by default
// Use @Qualifier to override and pick StripePaymentService
```

💻 @Autowired on Constructor, Setter, Field

```
@Service
public class OrderService {

    private final PaymentService paymentService;
    private EmailService emailService;
    @Autowired private SMSService smsService;   // field injection (avoid)

    // Constructor injection
    @Autowired
    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }

    // Setter injection — optional dependency
    @Autowired(required = false)  // won't fail if EmailService bean missing
    public void setEmailService(EmailService emailService) {
        this.emailService = emailService;
    }
}
```

❓ **Interview Q:** What happens if two beans of the same type exist? How does Spring decide?

✅ Spring throws NoUniqueBeanDefinitionException — doesn't know which to pick.
✅ Solution 1: @Primary on one implementation → becomes the default choice.
✅ Solution 2: @Qualifier('beanName') at the injection point → explicitly choose.
✅ Solution 3: Name the field same as the bean ID → Spring matches by name.
✅ Best practice: Use @Qualifier for clarity. Avoid relying on field name matching.

🔟 Spring Configuration Styles

Spring has evolved over the years. There are 3 ways to configure it. Modern Spring Boot uses annotation + Java config.

📄 Style 1: XML Configuration (Legacy)

```
<!-- applicationContext.xml -->
<beans xmlns="http://www.springframework.org/schema/beans">

    <!-- Constructor Injection in XML -->
    <bean id="orderService" class="com.app.OrderService">
        <constructor-arg ref="paymentService"/>
    </bean>

    <!-- Setter Injection in XML -->
    <bean id="restaurantService" class="com.app.RestaurantService">
        <property name="repository" ref="restaurantRepo"/>
        <property name="maxResults" value="20"/>
    </bean>

    <bean id="paymentService" class="com.app.RazorpayPaymentService"/>
    <bean id="restaurantRepo" class="com.app.RestaurantRepository"/>
</beans>

// Load it:
ApplicationContext ctx = new ClassPathXmlApplicationContext("applicationContext.xml");
```

✅ Style 2: Annotation + @ComponentScan (Modern, Most Common)

```
@Configuration
@ComponentScan(basePackages = "com.foodieexpress")  // scan this package
public class AppConfig {
    // No bean definitions needed here!
    // Spring auto-discovers @Component, @Service, @Repository classes
}

@Service   // auto-detected by @ComponentScan
public class RestaurantService {
    @Autowired
    private RestaurantRepository repository;
}

@Repository  // auto-detected
public class RestaurantRepository { }
```

☕ Style 3: Java @Configuration + @Bean (For Third-Party / Complex Setup)

```
@Configuration
public class InfrastructureConfig {

    // Register a third-party class (can't add @Service to it)
    @Bean
    public ObjectMapper objectMapper() {
        ObjectMapper mapper = new ObjectMapper();
        mapper.configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, false);
        return mapper;
    }

    // Bean with external config value
    @Bean
    @ConfigurationProperties(prefix = "payment")
    public PaymentConfig paymentConfig() {
        return new PaymentConfig();
```

```
    }

    // Bean depends on another bean
    @Bean
    public PaymentService paymentService(PaymentConfig config) { // injected by Spring
        return new RazorpayPaymentService(config.getApiKey());
    }
}
```

❓ **Interview Q:** What is @Configuration? How is it different from @Component?

✅ @Configuration marks a class as a source of bean definitions (@Bean methods).
✅ @Component marks a class as a generic Spring bean (auto-detected by @ComponentScan).
✅ Key difference: @Configuration uses CGLIB proxy — calling a @Bean method twice returns the SAME instance (singleton). @Component does not proxy.
✅ Use @Configuration when you need to define beans manually (especially third-party).
✅ Use @Component/@Service/@Repository for your own classes.

1️⃣1️⃣ Key Annotations — Complete Reference

🏷️ Stereotype Annotations

Annotation	Purpose
@Component	Generic Spring-managed bean. Base for all stereotypes.
@Service	@Component + signals 'business logic layer'. No extra behavior.
@Repository	@Component + translates JPA/SQL exceptions to Spring's DataAccessException.
@Controller	@Component + marks web MVC controller (returns view name).
@RestController	@Controller + @ResponseBody. Returns JSON/XML directly.
@Configuration	@Component + CGLIB proxy. Used to define @Bean methods.

🔗 Injection & Wiring Annotations

Annotation	Purpose
@Autowired	Tells Spring to inject a matching bean (by type, then name).
@Qualifier("name")	Specify exact bean name when multiple beans of same type exist.
@Primary	Mark one bean as the preferred choice when multiple exist.
@Value("${prop}")	Inject a value from application.properties into a field.

Annotation	Purpose
@Inject	JSR-330 standard equivalent of @Autowired (works the same).
@Resource	JSR-250. Injects by NAME first, then by type. From javax package.

♻️ Lifecycle & Scope Annotations

Annotation	Purpose
@PostConstruct	Runs after bean is created and all dependencies injected. Init logic.
@PreDestroy	Runs before bean is destroyed. Cleanup logic.
@Scope("prototype")	Creates a new instance every time the bean is requested.
@Lazy	Delays bean creation until first use (overrides eager default).
@DependsOn("other")	Ensures 'other' bean is created BEFORE this bean.

⚙️ Context & Config Annotations

Annotation	Purpose
@ComponentScan	Tells Spring which packages to scan for @Component classes.
@PropertySource	Load a .properties file: @PropertySource('classpath:app.properties')
@Import	Import another @Configuration class into the current one.
@Profile("dev")	Bean is only created when the 'dev' profile is active.
@Conditional	Bean created only if a specific condition is true.
@Bean(name="id")	Custom bean name. Default = method name.

❓ **Interview Q:** What is @Value annotation? How do you use it?

✅ @Value injects values from application.properties or environment variables into fields.
✅ Example in application.properties: payment.api.key=rzp_live_abc123
✅ In Java: @Value("${payment.api.key}") private String apiKey;
✅ Default value: @Value("${timeout:5000}") private int timeout; // 5000 if not set
✅ SpEL expression: @Value("#{2 * 3}") private int result; // injects 6

1️⃣2️⃣ @ComponentScan — How Spring Finds Your Beans

@ComponentScan tells Spring: 'Go look in these packages and find all classes annotated with @Component, @Service, @Repository, @Controller — and register them as beans.'

```
// Basic usage — scans the current package and all sub-packages
@Configuration
@ComponentScan(basePackages = "com.foodieexpress")
public class AppConfig { }

// Multiple packages
@ComponentScan(basePackages = {"com.foodieexpress.service", "com.foodieexpress.repo"})

// Exclude certain classes
@ComponentScan(
    basePackages = "com.foodieexpress",
    excludeFilters = @ComponentScan.Filter(
        type = FilterType.ANNOTATION, classes = Controller.class
    )
)

// Spring Boot: @SpringBootApplication already includes @ComponentScan
// It scans the package where @SpringBootApplication class is located + sub-packages
@SpringBootApplication  // = @Configuration + @EnableAutoConfiguration + @ComponentScan
public class FoodieExpressApp { }
```

💡 **Remember:** In Spring Boot, always place your main class at the ROOT package (com.foodieexpress). This ensures @ComponentScan finds ALL your classes in sub-packages automatically.

❓ **Interview Q:** What is @ComponentScan? What happens if you don't add it?

✅ @ComponentScan tells Spring which packages to scan for annotated beans.
✅ Without it: Spring finds NO beans → NoSuchBeanDefinitionException everywhere.
✅ Spring Boot's @SpringBootApplication includes @ComponentScan automatically.
✅ It scans the package of the main class AND all its sub-packages.
✅ If you move main class to wrong package, beans in other packages won't be found!

1️⃣3️⃣ AOP — Aspect-Oriented Programming (Basics)

AOP lets you add cross-cutting concerns (logging, security, transactions) WITHOUT modifying business logic code. Think of it as a plugin system.

🎭 **Real-World Analogy**

🔍 Without AOP: You add logging code inside every method of every service. 100 methods = 100 places!

🔌 With AOP: Write logging logic ONCE in an Aspect. Spring automatically applies it to all methods.

Like a CCTV camera — every room (method) is monitored without you installing a camera in each room.

📖 AOP Key Terms

Term	Simple Meaning
Aspect	A class that contains cross-cutting logic (e.g., LoggingAspect, SecurityAspect)
Advice	The actual code that runs (what to do). Types: @Before, @After, @Around, @AfterReturning, @AfterThrowing
JoinPoint	A point in your program where advice can be applied (method execution, field access)
Pointcut	Expression that selects which JoinPoints to apply the advice to
Weaving	Process of applying aspects to target objects. Spring does runtime proxy weaving.

```java
@Aspect
@Component
public class LoggingAspect {

    // Pointcut: match ALL methods in service package
    @Before("execution(* com.foodieexpress.service.*.*(..))")
    public void logBefore(JoinPoint joinPoint) {
        System.out.println("Calling: " + joinPoint.getSignature().getName());
    }

    // Runs AFTER method returns successfully
    @AfterReturning(pointcut = "execution(* com.foodieexpress.service.*.*(..))",
                    returning = "result")
    public void logAfter(JoinPoint jp, Object result) {
        System.out.println("Returned: " + result);
    }

    // Around — most powerful: runs before AND after, can modify return value
    @Around("execution(* com.foodieexpress.service.*.*(..))")
    public Object measureTime(ProceedingJoinPoint pjp) throws Throwable {
        long start = System.currentTimeMillis();
        Object result = pjp.proceed();  // call the actual method
        long time = System.currentTimeMillis() - start;
        System.out.println("Execution time: " + time + "ms");
        return result;
    }
}
```

❓ **Interview Q:** What is AOP? What are the types of Advice?

✅ AOP = separate cross-cutting concerns from business logic.
✅ Cross-cutting concerns: logging, security, transactions, caching, performance tracking.
✅ 5 types of Advice:
@Before — runs BEFORE the method
@After — runs AFTER (always, even if exception)
@AfterReturning — runs after SUCCESSFUL return
@AfterThrowing — runs only if exception is thrown

@Around — wraps the method, most powerful, can change return value
✅ Real use in Spring: @Transactional uses AOP to wrap methods with transaction logic!

1️⃣4️⃣ Spring Profiles — Environment-Specific Config

Profiles let you have different configurations for different environments (dev, test, prod) without changing code.

🎭 **Real-World Analogy**
🏠 Analogy: You have different menus for Breakfast, Lunch, Dinner (different 'profiles').
Same restaurant (app), but different items available based on the time (active profile).
Dev profile: use H2 in-memory DB. Prod profile: use MySQL. Test profile: use mock data.

```
// application.properties — set active profile
spring.profiles.active=dev

// application-dev.properties
spring.datasource.url=jdbc:h2:mem:testdb

// application-prod.properties
spring.datasource.url=jdbc:mysql://prod-server:3306/foodieexpress

// Profile-specific bean
@Service
@Profile("dev")
public class MockPaymentService implements PaymentService {
    // Used only in dev — always returns success
}

@Service
@Profile("prod")
public class RazorpayPaymentService implements PaymentService {
    // Real payment gateway — only in production
}

// Multiple profiles
@Profile({"dev", "test"})   // bean active in BOTH dev and test

// NOT a profile
@Profile("!prod")           // bean active in ALL profiles EXCEPT prod
```

❓ **Interview Q:** What are Spring Profiles? How do you activate them?

✅ Profiles = way to have different beans/config for different environments.
✅ Use @Profile('dev') on a bean class or @Bean method.
✅ Activate via: application.properties → spring.profiles.active=dev
✅ Or JVM arg: -Dspring.profiles.active=prod
✅ Or environment variable: SPRING_PROFILES_ACTIVE=prod
✅ You can also have profile-specific property files: application-dev.properties

✅ Common use: Different DB URL, API keys, logging levels per environment.

1️⃣5️⃣ Top 15 Interview Q&A — Quick Revision

❓ **Interview Q:** What is tight coupling and how does Spring solve it?

✅ Tight coupling = Class A creates Class B inside itself (new B()). Changing B breaks A.
✅ Spring solves this with DI: A just declares 'I need a B'. Spring creates B and gives it.
✅ Result: A doesn't know HOW B is created → loose coupling → easier to test and change.

❓ **Interview Q:** What is the Spring IoC Container?

✅ IoC Container = Spring's core engine that creates and manages all beans (objects).
✅ You give it configuration (XML/annotations). It creates objects and wires them together.
✅ Two containers: BeanFactory (basic) and ApplicationContext (advanced — use this).
✅ ApplicationContext is created when your Spring app starts.

❓ **Interview Q:** How does @Autowired work internally?

✅ At startup, Spring scans all beans. It processes @Autowired using AutowiredAnnotationBeanPostProcessor.
✅ It finds the field/constructor/setter with @Autowired, looks up the correct bean by type.
✅ If one match → injects it. Multiple → checks @Qualifier/@Primary. None → exception.
✅ This happens AFTER the bean is instantiated (constructor called) but before @PostConstruct.

❓ **Interview Q:** What is the difference between @Autowired and @Inject?

✅ @Autowired = Spring-specific annotation. Has 'required' attribute (required=false allowed).
✅ @Inject = JSR-330 standard (javax.inject). Works the same way. No 'required' attribute.
✅ @Autowired is more commonly used in Spring projects.
✅ @Inject works with both Spring and other DI frameworks (like CDI in Jakarta EE).

❓ **Interview Q:** Can you have a circular dependency in Spring? How to solve it?

✅ Circular dependency = A depends on B and B depends on A.
✅ With constructor injection: Spring CANNOT resolve this → BeanCurrentlyInCreationException.
✅ With field/setter injection: Spring can resolve using a proxy (but it's a bad design smell).
✅ Better Solutions:

1. Refactor — extract shared logic into a third class C
2. Use @Lazy on one of them — delays creation until first use
3. Use an event-driven approach (Spring ApplicationEvent)

❓ **Interview Q:** What is BeanPostProcessor?

✅ BeanPostProcessor = a Spring hook to modify beans AFTER they are created but BEFORE they are used.
✅ It has 2 methods: postProcessBeforeInitialization() and postProcessAfterInitialization()
✅ Spring itself uses it internally: AutowiredAnnotationBeanPostProcessor (processes @Autowired).
✅ You can write custom BeanPostProcessor to add logging, proxies, or validation to all beans.

❓ **Interview Q:** What is Spring's proxy mechanism? How does @Transactional work internally?

✅ Spring creates a PROXY (wrapper) around your bean at startup for AOP features.
✅ @Transactional uses AOP proxy: when you call a @Transactional method, the proxy intercepts it, starts a transaction, calls your method, commits/rolls back.
✅ Two proxy types: JDK Dynamic Proxy (for interfaces) and CGLIB Proxy (for classes).
✅ Important: @Transactional does NOT work when you call a @Transactional method from WITHIN the same class (self-invocation skips the proxy)!

📋 Final Cheat Sheet — Everything at a Glance

⚡ One-Line Summary of Every Key Concept

Concept	One-Line Meaning
Spring Framework	Java framework that removes boilerplate and makes apps testable/maintainable.
IoC	Control of object creation transferred from your code to Spring.
DI	Spring injects dependencies instead of you creating them with 'new'.
IoC Container	Spring's factory that creates, wires, and manages all beans.

Concept	One-Line Meaning
BeanFactory	Basic IoC container. Lazy init. Rarely used directly.
ApplicationContext	Advanced IoC container. Eager init. Always use this.
Bean	Any Java object managed by Spring's IoC container.
Singleton Scope	Default. ONE instance shared across the whole application.
Prototype Scope	NEW instance every time the bean is requested.
@PostConstruct	Init logic that runs after all dependencies are injected.
@PreDestroy	Cleanup logic that runs before the bean is destroyed.
@Autowired	Ask Spring to inject the right bean here.
@Qualifier	Tell Spring WHICH bean to inject when multiple exist.
@Primary	Mark one bean as default when multiple of same type exist.
@ComponentScan	Tell Spring which packages to scan for bean classes.
@Configuration + @Bean	Manually define beans, especially for third-party classes.
AOP	Add cross-cutting logic (logging, tx) without touching business code.
@Profile	Activate beans only in specific environments (dev/prod/test).

🎯 Interview Golden Rules

1. Always answer with: Concept → Simple Words → Real Example
2. Use the Food Ordering App (FoodieExpress) as your example project
3. Connect Spring Core → IoC → DI → Beans → Lifecycle → AOP
4. Always say: 'I prefer Constructor Injection because...'
5. **ApplicationContext > BeanFactory. Singleton is default scope.** 🚀

All the best! You've got this! 💪

Spring Core & IoC Container — Complete Notes